AULA 3 e 4 · DESENVOLVIMENTO DE SOFTWARE CLIENTE-SERVIDOR

Aplicações Cliente-Servidor

Comunicação em rede pela ótica da aplicação: modelo cliente-servidor, sockets e uma POC em Java



CONTEXTO DA DISCIPLINA

Onde estamos na disciplina



Aulas 1 e 2

Redes de computadores

Intercomunicação: como os dados viajam entre máquinas.

Concluído



Aula 3 · hoje

Aplicações em rede

Modelo cliente-servidor, sockets e uma POC em Java.

Hoje



Aula 4

Evolução da aplicação

Concorrência, protocolos mais ricos e interface gráfica.

Próxima

OBJETIVOS

O que você vai aprender hoje

1. Explicar o modelo cliente-servidor e o papel de cada lado
2. Descrever o que é um socket e como ele identifica uma comunicação
3. Distinguir TCP e UDP do ponto de vista do desenvolvedor
4. Ler e executar uma aplicação Java simples com Socket e ServerSocket



AO FINAL DA AULA

Um cliente e um servidor Java conversando por socket

ROTEIRO

Agenda da aula

01

Modelo cliente-servidor

Papéis, ciclo requisição-resposta, exemplos e arquiteturas.

Slides 5 a 11

02 e 03

Sockets e comunicação

Portas, TCP e UDP, classes Java e protocolos de aplicação.

Slides 12 a 20

03 e 04

Prática: POC em Java

Um servidor e um cliente de console, do código à execução.

Slides 21 a 28

Fechamento: resumo, limitações da POC e atividade proposta (slides 29 e 30)

Ponto de partida: o que a rede já nos dá

Aplicação
HTTP, SMTP, o seu protocolo

Transporte
TCP, UDP, portas

Rede
IP, roteamento

Enlace e física
quadros, sinais, meio

A rede já resolve

- Endereçar máquinas (IP)
- Entregar dados a um processo (portas, TCP e UDP)
- Transportar bits pelo meio físico

Falta à aplicação

- Definir o que as mensagens significam
- Decidir quem inicia e quem responde
- Escrever o código que usa a rede (sockets)

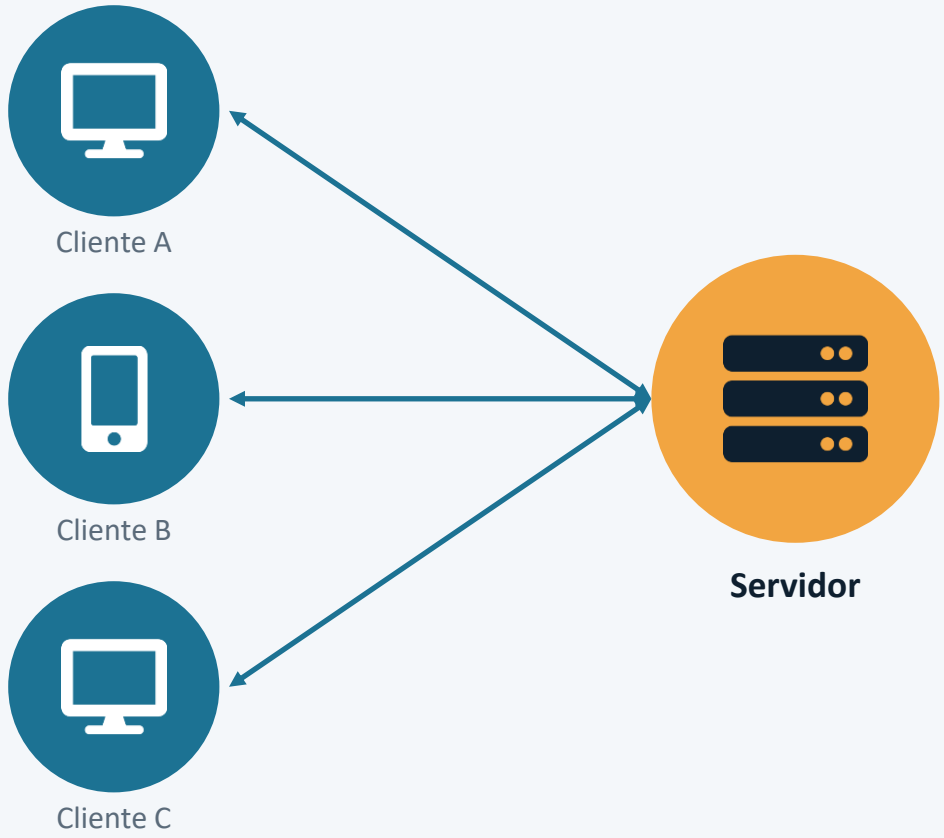
O que é o modelo cliente-servidor

Um modelo de distribuição de tarefas entre processos que se comunicam pela rede.

Servidor: oferece um serviço e aguarda pedidos

Cliente: solicita o serviço e usa a resposta

Papéis de processos, não de máquinas: o mesmo computador pode ter os dois



Papéis: cliente e servidor



Cliente

- Inicia a comunicação (papel ativo)
- Conhece o endereço e a porta do servidor
- Vida curta: conecta, pede, recebe e encerra
- Costuma interagir com o usuário

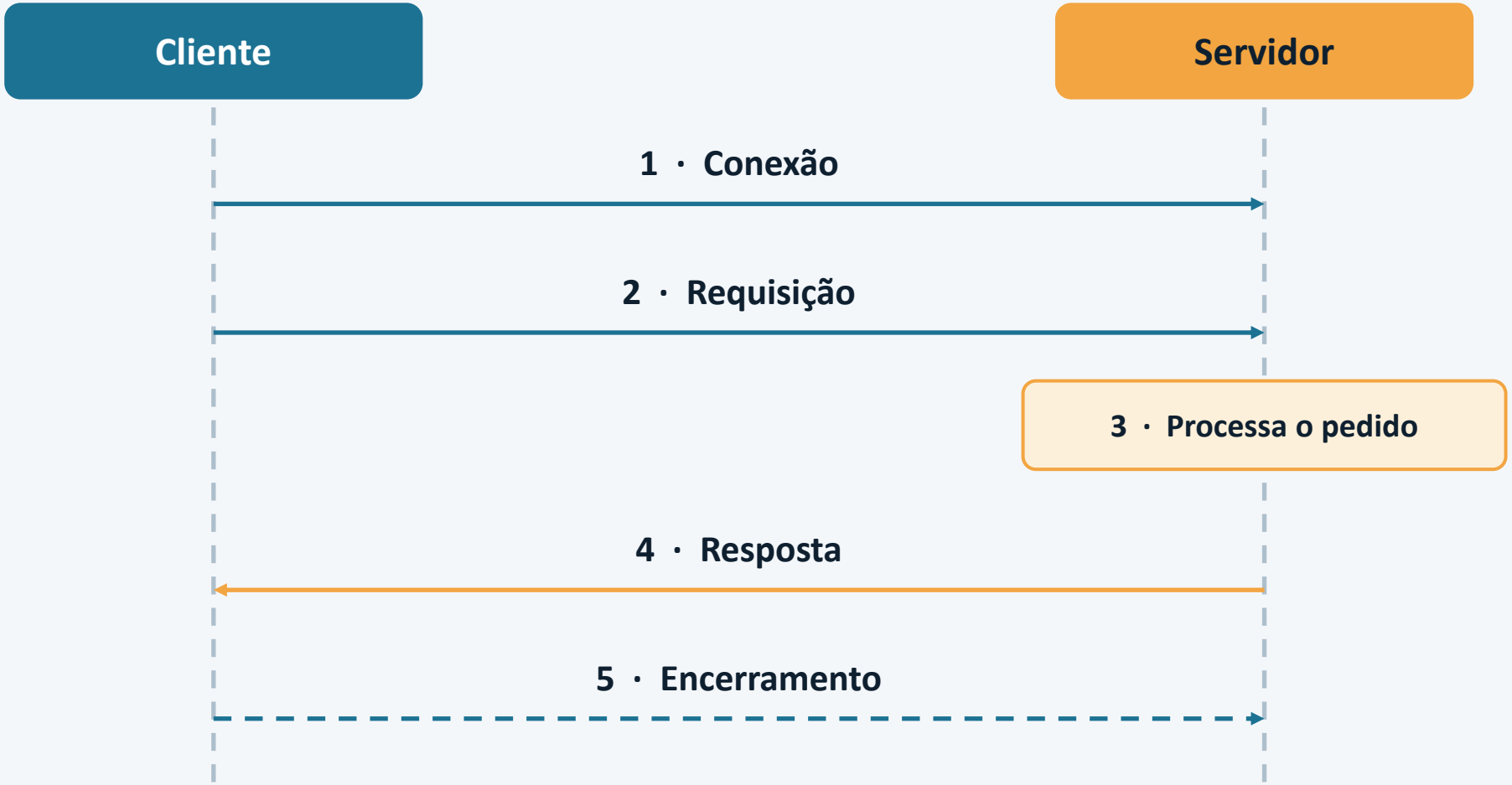


Servidor

- Aguarda conexões (papel passivo)
- Tem endereço e porta conhecidos
- Executa continuamente, à espera de pedidos
- Atende um ou vários clientes

Um mesmo programa pode ser cliente de um serviço e servidor de outro.

O ciclo requisição-resposta



Exemplos do dia a dia



Web

Navegador ↔ servidor web (HTTP e HTTPS)



E-mail

Cliente de e-mail ↔ servidor (SMTP, IMAP)



Banco de dados

Aplicação ↔ SGBD (por exemplo, via JDBC)



Jogos online

Jogo no dispositivo ↔ servidor da partida



Apps móveis

Aplicativo ↔ API hospedada no servidor

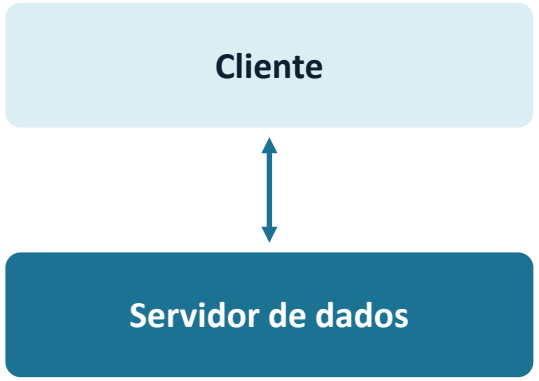


Acesso remoto

Terminal ↔ servidor SSH

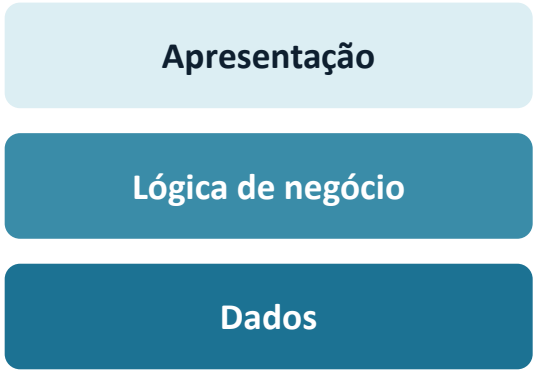
Arquiteturas: duas camadas, três camadas e P2P

Duas camadas



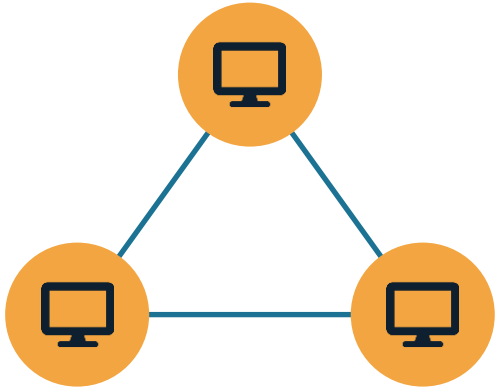
Ex.: aplicação desktop acessando o banco de dados diretamente.

Três camadas



Ex.: app web ou móvel que chama uma API, que acessa o banco de dados.

Ponto a ponto (P2P)



Cada nó é cliente e servidor ao mesmo tempo. Ex.: compartilhamento de arquivos.

Desafios de quem constrói aplicações em rede



Concorrência

Vários clientes ao mesmo tempo disputando o servidor.



Falhas de rede

Conexões caem e mensagens atrasam ou se perdem.



Estado da sessão

Lembrar (ou não) quem é o cliente entre uma requisição e outra.

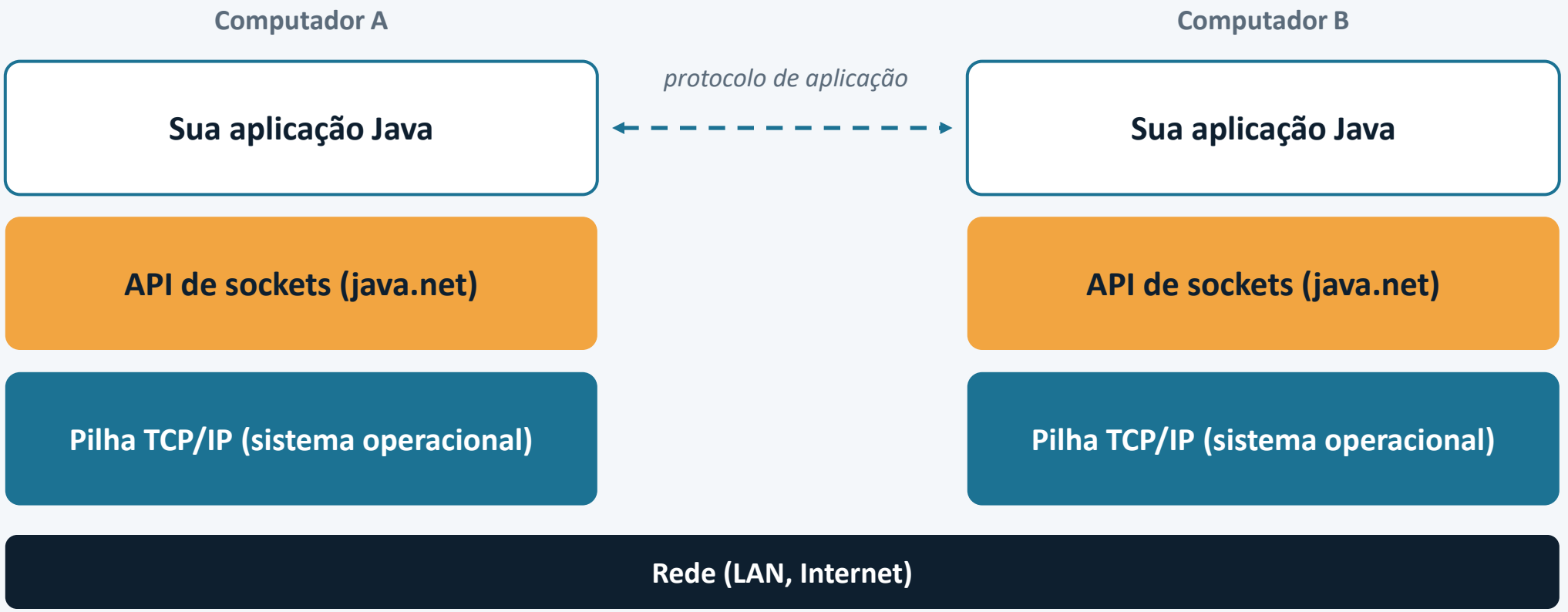


Segurança

Autenticação, criptografia e validação do que chega.

Hoje: uma conversa simples entre dois programas. Na aula 4: enfrentar esses desafios.

Da rede à aplicação: onde os sockets entram



O socket é a porta de entrada da aplicação para os serviços de rede do sistema operacional.

O que é um socket

DEFINIÇÃO

Socket é um ponto de extremidade de uma comunicação em rede.

endereço IP + porta + protocolo

Analogia

IP: o endereço do prédio

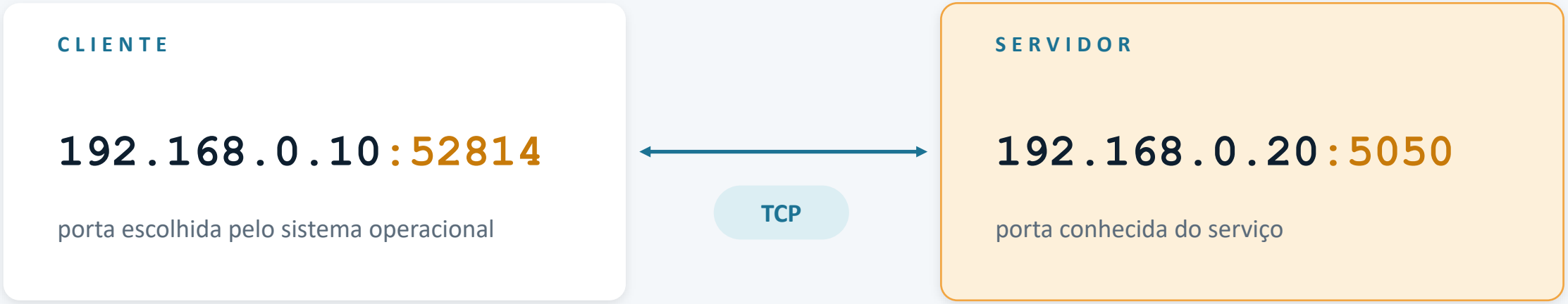
Porta: o número do apartamento

Socket: o ponto onde a conversa acontece

Para o programador

Um socket se comporta como um arquivo: abrir, ler, escrever e fechar.

Como uma conexão é identificada



Uma conexão TCP é identificada por cinco valores:

Protocolo | **IP de origem** | **Porta de origem** | **IP de destino** | **Porta de destino**

Dois clientes falando com o mesmo servidor geram conexões distintas: pelo menos um dos valores de origem muda.

Portas: o número do apartamento

0 – 1023

Bem conhecidas

80 HTTP
443 HTTPS
22 SSH
25 SMTP

Em muitos sistemas exigem privilégios de administrador.

1024 – 49151

Registradas

3306 MySQL
5432 PostgreSQL
8080 HTTP alternativa

Livres para aplicações comuns.

49152 – 65535

Dinâmicas (efêmeras)

Porta de origem temporária do cliente

Atribuídas pelo sistema operacional; a faixa exata varia.

Na POC: porta 5050, na faixa registrada e sem necessidade de privilégios.

TCP × UDP: duas formas de conversar

TCP *como uma ligação telefônica*

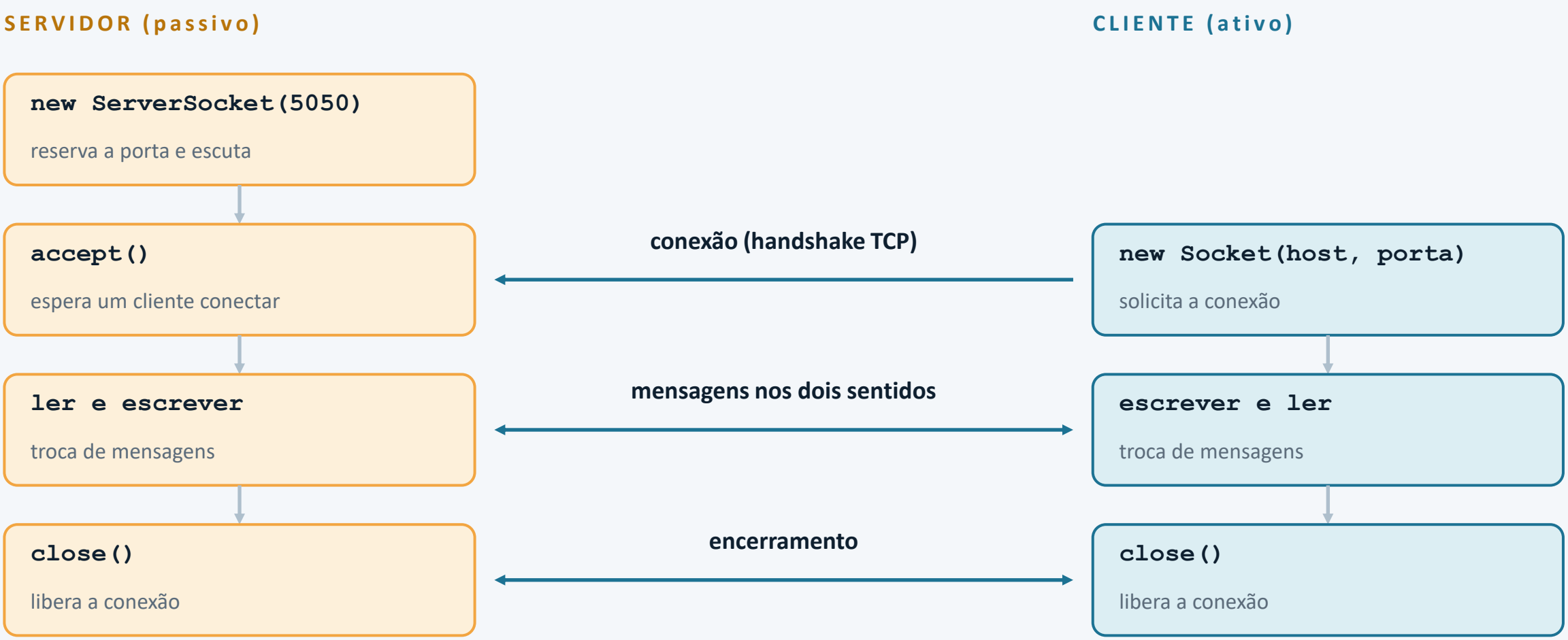
- Orientado a conexão
- Entrega confiável e em ordem
- Controle de fluxo e de congestionamento
- Java: Socket e ServerSocket

UDP *como enviar cartões-postais*

- Sem conexão
- Sem garantia de entrega ou de ordem
- Menos sobrecarga e menor latência
- Java: DatagramSocket e DatagramPacket

Nesta aula: TCP, pela confiabilidade e pela simplicidade de uso.

Ciclo de vida de uma conexão TCP



Sockets em Java: as classes principais

ServerSocket

TCP · servidor

Escuta em uma porta. O método accept() bloqueia até um cliente chegar e devolve um Socket.

Socket

TCP · cliente e servidor

Extremidade de uma conexão. Oferece getInputStream() e getOutputStream().

InetAddress

endereços

Representa um endereço IP ou nome de host. Ex.: InetAddress.getByName("localhost").

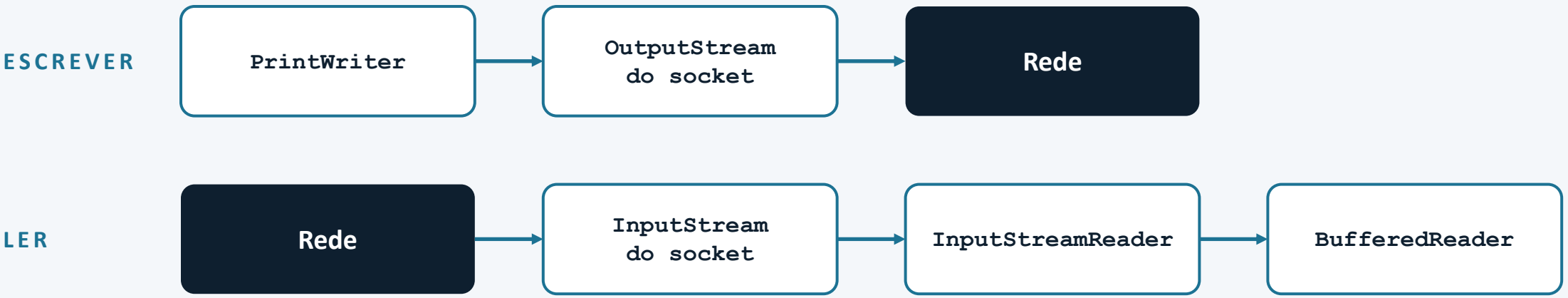
DatagramSocket

UDP · sem conexão

Envia e recebe datagramas (DatagramPacket) sem estabelecer conexão.

Pacotes: java.net (rede) e java.io (fluxos de entrada e saída)

Lendo e escrevendo: streams sobre o socket



`println(msg)` envia uma linha; com autoflush, sai na hora
`readLine()` bloqueia até chegar uma linha; retorna null se o outro lado fechou
`try-with-resources` fecha streams e socket automaticamente

Protocolo de aplicação: dando sentido aos bytes

O socket entrega bytes, não significado. O protocolo precisa responder:

1. Quem fala primeiro?
2. Qual é o formato das mensagens?
3. Como saber onde uma mensagem termina?
4. Como e quando encerrar a conversa?

Protocolo de aplicação: conjunto de regras que responde a essas perguntas.

EXEMPLO: HTTP

```
Cliente:
  GET /index.html HTTP/1.1
  Host: www.exemplo.com.br

Servidor:
  HTTP/1.1 200 OK
  Content-Type: text/html
  Content-Length: 1256
```

Regras claras permitem que programas escritos por pessoas diferentes se entendam.

A POC: escopo e funcionalidades

Comandos disponíveis

- `ECO <msg>`
- `HORA`
- `SOMA <a> <b>`
- `MAIOR <a> <b>`
- `PAR <n>`
- `SAIR`

Dentro do escopo

- Vários clientes ao mesmo tempo (threads)
- Validação de comandos e de números
- Interface gráfica em JavaFX
- Protocolo de texto, uma mensagem por linha

Fora do escopo

- Persistência e banco de dados
- Autenticação e criptografia
- Reconexão automática do cliente
- Testes automatizados

O protocolo da POC

Comando	Retorno em caso de sucesso
`ECO <mensagem>`	a mensagem em maiúsculas
`HORA`	hora atual do servidor
`SOMA <a> <b>`	a soma dos dois números
`MAIOR <a> <b>`	o maior dos dois números
`PAR <n>`	PAR ou IMPAR
`SAIR`	TCHAU, e a conexão é encerrada

Qualquer entrada inválida — comando desconhecido, argumentos faltando, número mal formatado — devolve uma linha **`ERRO <motivo>`**, e a conexão continua aberta para o próximo comando.

Código: aceitar vários clientes

```java
public class Servidor {
    public static void main(String[] args) {
        try (ServerSocket servidor =
                 new ServerSocket(5050)) {
            while (true) {
                Socket s = servidor.accept();
                int id = CONTADOR.incrementAndGet();
                new Thread(new ClienteHandler(s, id)).start();
            }
        } catch (IOException e) {
            log("Erro: " + e.getMessage());
        }
    }
}
class ClienteHandler implements Runnable {
    public void run() {
        // abre entrada/saida do socket ...
        String linha;
        while ((linha = entrada.readLine()) != null) {
            String resposta = processar(linha.trim());
            saida.println(resposta);
            if (resposta.equals("OK SAIR TCHAU")) break;
        }
    }
}
```

ARQUIVO

Servidor.java

EXECUTAR

java Servidor

NOVIDADE

uma thread por cliente

Validação: do texto ao comando executado

```
private String processar(String linha) {
    if (linha.isEmpty()) {
        return "ERRO COMANDO_VAZIO";
    }
    String[] partes = linha.split("\\s+");
    String comando = partes[0].toUpperCase();
    try {
        switch (comando) {
            case "SOMA":
                if (partes.length != 3)
                    return "ERRO ARGUMENTOS_INVALIDOS";
                double a = Double.parseDouble(partes[1]);
                double b = Double.parseDouble(partes[2]);
                return "OK SOMA " + (a + b);
            // ECO, HORA, MAIOR, PAR, SAIR: mesmo padrao
            default:
                return "ERRO COMANDO_DESCONHECIDO "
                        + comando;
        }
    } catch (NumberFormatException e) {
        return "ERRO NUMERO_INVALIDO";
    }
}
```

1 **Comando vazio**

linha.isEmpty() intercepta antes de tentar interpretar qualquer coisa.

2 **Argumentos faltando**

partes.length é conferido antes de acessar partes[1] ou partes[2].

3 **Número mal formatado**

Double.parseDouble lança NumberFormatException, capturada uma única vez para todos os comandos.

4 **Comando desconhecido**

o default do switch cobre qualquer palavra que não seja um comando válido.

Cliente gráfico: JavaFX



Executando a POC

TERMINAL · SERVIDOR

```
$ java Servidor
Servidor aguardando conexoes
na porta 5050...

[20:28] Cliente #1 conectado
[20:28] Cliente #2 conectado
```

CLIENTE GRÁFICO (JAVAFX)

```
$ mvn javafx:run

# ou, com o SDK baixado:
$ java --module-path SDK/lib \
    --add-modules javafx.controls \
    ClienteGUI
```

1. **Iniciar o servidor (ele espera)**
2. **Abrir dois ou mais clientes JavaFX**
3. **Testar comandos e validações**

O Servidor.java não depende do JavaFX: continua rodando com "java Servidor", sem SDK adicional.

Demonstração ao vivo e observação da rede

Roteiro

1. Iniciar o servidor e abrir dois clientes JavaFX
2. Testar ECO, HORA, SOMA, MAIOR e PAR em cada um
3. Digitar letra num campo numérico: o campo bloqueia
4. Enviar BANANA no comando livre: ver o ERRO
5. Encerrar um cliente com SAIR; o outro continua
6. Fechar o servidor: o cliente perde a conexão

OBSERVANDO A REDE

```
# duas conexoes estabelecidas
netstat -an | findstr 5050    (Windows)
ss -tan | grep 5050           (Linux)
lsof -i :5050                 (macOS)

# cliente alternativo, texto puro
nc localhost 5050

# Wireshark, interface loopback
tcp.port == 5050
```

Conexão com as aulas 1 e 2: observe duas conexões TCP simultâneas.

Limitações da POC



Sem persistência

Nada é salvo: cada conexão nova começa do zero, sem histórico.



Sem segurança

Texto puro na rede, sem autenticação nem criptografia.



Sem reconexão automática

Se a conexão cair, o cliente precisa reconectar manualmente.



Protocolo ainda simples

Sem versionamento nem comandos compostos; uma linha por vez.

Cada limitação restante continua sendo um ponto de partida para a aula 4.

FECHAMENTO

Resumo da aula

- **Cliente-servidor:** processos com papéis distintos, quem pede e quem atende.
- **Socket:** extremidade de comunicação, identificada por IP e porta.
- **TCP e UDP:** conexão confiável ou datagramas leves.
- **Em Java:** ServerSocket, Socket e streams de entrada e saída.
- **Protocolo de aplicação:** as regras que dão sentido aos bytes.

PARA LEVAR

Socket =
IP + porta.

O resto é protocolo de aplicação.

FECHAMENTO

Próximos passos

ATIVIDADE SUGERIDA

- Executar a POC em duas máquinas da mesma rede
- Acrescentar o comando HORA: o servidor devolve a hora atual
- Capturar o tráfego no Wireshark e localizar o handshake TCP

Leituras: Kurose e Ross, Redes de Computadores e a Internet, capítulo sobre a camada de aplicação (programação de sockets) · Oracle Java Tutorials, trilha Custom Networking